

CSCE 676 DATA MINING · MODULE 10 · LECTURE NOTES

Scale and Systems

What distribution actually costs.

MODULE OUTCOMES

Explain the shuffle and why it dominates cost.

Explain the MapReduce and dataflow models.

Explain skew and how it is handled.

Explain when a single machine is the right answer.

Design a computation for the cost model that applies.

1 The cost model

THE SHUFFLE IS THE EXPENSIVE OPERATION, AND EVERYTHING ELSE IS ARRANGED AROUND IT

A distributed computation is cheap when every machine works independently on its own partition, and expensive when data has to move between machines — and that movement is the shuffle, which involves serialisation, network transfer, and frequently a disk write.

So the design question is how many shuffles a computation requires — map-only work is nearly free, and anything requiring a global grouping, a join on a non-partitioning key, or a total sort costs a full movement of the data.

Which means the goal is to reduce the data before shuffling it: combine locally before the shuffle, filter as early as possible, project away columns you do not use, and broadcast a small side input rather than performing a join — four techniques that between them handle most cases.

And it is Module 01 §2's cost model arriving again — passes became shuffles, and the arithmetic is still not the constraint. The through-line of this course is that the obvious cost model is the wrong one at every scale.

MAPREDUCE

map -> shuffle -> reduce, with the intermediate result written to disk. Simple, fault-tolerant by re-execution, and slow for iterative work because every round pays the full disk cost again.

DATAFLOW (Spark and its successors)

a DAG of transformations over partitioned collections, kept in memory where possible. Lineage-based recovery rather than replication. Much better for iterative work -- which is most of this course's algorithms.

VERTEX-CENTRIC (Pregel)

Module 07's model: each vertex computes and sends messages, synchronised in supersteps. Fits graph algorithms well and very little else.

AND THE SHARED ASSUMPTION

the computation decomposes into independent work on partitions, with occasional communication. Anything genuinely sequential does not fit any of them.

The shared assumption is the limit, and naming it is worth doing: **a genuinely sequential algorithm does not become parallel by being expressed in one of these models**. An algorithm whose step $n+1$ depends on the complete result of step n will run with one machine working and the rest idle, whatever framework it is written in — **which prevents the expectation that any algorithm distributes** and saves a good deal of disappointment.

2 Skew

ONE SLOW PARTITION SETS THE RUNTIME, AND HEAVY-TAILED DATA GUARANTEES ONE

A stage finishes when its slowest task finishes — so **a partition holding ten times the data takes roughly ten times as long, and every other machine sits idle waiting for it**. The parallelism is lost entirely to the one key.

And the data is heavy-tailed (Module 07 §1): **one key with a hundred million records alongside a million keys with ten each is the normal case** in real data, not a pathological one you can design around.

So the fixes are specific and worth knowing: salt the hot key across several partitions and combine the partial results afterwards; broadcast the small side of a skewed join rather than shuffling both; or detect the heavy keys and handle them in a separate code path.

And diagnose it by looking at the task duration distribution within a stage — **a long tail there is skew, and an average task time tells you nothing at all about it, which is why aggregate metrics mislead here** exactly as they do in CSCE 638 Module 12's disaggregation argument.

- **The task duration distribution per stage**, not the mean — **which is where skew is visible and nowhere else**, and which every framework's interface will show you if you look.
- **The shuffle read and write volumes**, which tell you directly whether you are moving more data than the computation needs — and a shuffle larger than the input is a signal.
- **The number of stages**, since **each stage boundary is a shuffle** — and a query plan with eight stages is doing something that you may well be able to express in three.
- **Spill to disk**, which means a partition did not fit in the available memory and the stage is now paying disk costs it was supposed to avoid — usually fixed by more partitions rather than more memory.
- **And the input format — a columnar format with predicate pushdown frequently beats any amount of tuning**, by simply reading less data from disk. **Reading less is the first optimisation**, and it is **usually a format and projection decision rather than a parallelism one**.

3 When not to distribute

A SINGLE LARGE MACHINE BEATS A SMALL CLUSTER FOR A SURPRISING RANGE OF PROBLEMS

A commodity server now has hundreds of gigabytes of memory and dozens of cores — so a dataset of tens of gigabytes fits comfortably in memory with room to work, and an in-memory computation has no shuffle at all, which removes §1's entire cost.

And the published comparisons are uncomfortable: several well-known graph processing and mining benchmarks run faster on one well-programmed machine than on a modest cluster — because the coordination and communication overhead exceeds the parallelism gain at that scale, sometimes by a large factor.

So the first question to ask is simply: does it fit? And if it nearly fits, can sampling, column projection, or a sketch (Module 06) make it fit? — all three of which are cheaper than a cluster.

Which is worth asking because distribution has real costs that do not appear in a throughput number: operational complexity, much harder debugging, a far slower development loop, and a class of failure modes that do not exist on one machine (CSCE 678).

4 Designing for it

- The partitioning key, because it determines which operations are local and which require a shuffle — partition by the key you join and group on, and the shuffles disappear for those operations.
- The storage format and layout. Columnar, compressed, and partitioned on disk by the column you filter on — which is a one-time decision with a permanent effect on every query that follows, and is usually worth more than any runtime tuning.
- Where to approximate. A sketch or a sample that removes a shuffle is usually worth its error (Module 06) — and this is where this course's two halves meet: the approximation is chosen for a systems reason and its bound is still stated.
- And whether the algorithm is iterative, because that determines the engine — MapReduce is the wrong choice for anything with a loop, and most of this course's algorithms have loops.
- Then measure before tuning, because the bottleneck is rarely where you expect it — and the diagnostics in §2 take minutes while a speculative optimisation takes a day. The partitioning key is the highest-leverage decision: it converts shuffles into local work, and it is made once and then lived with.

AND THE HONEST FRAMING

Distribution buys you the ability to process data that does not fit on one machine, at the cost of communication, operational complexity, and a considerably slower development loop. That is the entire trade.

So use it when the data genuinely does not fit — and **check that claim rather than assuming it**, which is §3 and takes an afternoon to establish either way.

And when you do distribute, design around the shuffle — the partitioning key and the storage layout together determine most of the performance, before any configuration tuning is considered.

Which is a systems conclusion rather than an algorithmic one — and is **why CSCE 678 is a prerequisite for this module**: the partitioning, the failure model, and the consistency questions are all its material, arriving here with a data mining workload on top.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Mining of Massive Datasets, chapter 2 (free)	§1's models, with the communication cost analysis — and the replication-rate framework for designing MapReduce algorithms.
Zaharia et al. — Resilient Distributed Datasets (free)	§1's second model in the original — and the lineage recovery argument, which is the interesting part.
McSherry, Isard & Murray — Scalability! But at what COST? (free)	§3's argument, measured — the paper every student of this material should read before provisioning a cluster.
Spark's tuning and SQL performance documentation (free)	§2 and §4 practically — the skew handling and the partitioning guidance, from the implementation.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Count the shuffles** in a query plan you wrote.
- 2 **Rewrite it to use one fewer**, and measure the difference.
- 3 **Broadcast a small join side** and compare against the shuffled version.
- 4 **Construct a skewed key distribution** and observe the task duration tail.
- 5 **Salt the hot key** and measure the improvement.

- 6 **Convert a dataset to a columnar format** and measure the read volume for a filtered query.
- 7 **Run your whole computation on one large machine** and compare end to end against the cluster.
- 8 **Include the development time** in that comparison.
- 9 **Change the partitioning key** to match your join key and remeasure.
- 10 **Profile before and after one tuning change** and report whether the bottleneck was where you expected.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why is the shuffle the expensive operation?
- 2 Name four ways to reduce data before shuffling.
- 3 Name three programming models and what each suits.
- 4 What is their shared assumption, and what does it exclude?
- 5 Why does one slow partition set the runtime?
- 6 Why does heavy-tailed data guarantee skew, and what are the three fixes?
- 7 Where is skew visible, and where is it not?
- 8 Give five diagnostics, and the first optimisation.
- 9 Why might one machine beat a cluster?
- 10 Name four design decisions, and the highest-leverage one.